Module 3: Iterative Statements (Loops) in Java

Java Laboratory – Problem Statements

Module Overview

This module introduces students to iterative programming using Java loops. Students will learn how to execute a block of code repeatedly using `for`, `while`, and `do-while` loops. They will also explore nested loops, pattern generation, mathematical computations, menu-driven applications, and the use of `break` and `continue` statements. The exercises are designed to develop logical thinking, algorithmic skills, and efficient programming practices.

Total Practice Problems: 25

Difficulty Distribution:

- Beginner: 10 Problems
- Intermediate: 10 Problems
- Advanced: 5 Problems

Learning Objectives

After completing this module, students will be able to:

- Understand the purpose and syntax of iterative statements.
- Differentiate between `for`, `while`, and `do-while` loops.
- Solve repetitive computational problems using loops.
- Implement nested loops for pattern generation and matrix traversal.
- Control loop execution using `break` and `continue`.
- Design menu-driven programs using loops.
- Apply loops in real-world programming scenarios.

Concepts Covered

- `for` Loop
- `while` Loop

- `do-while` Loop
- Nested Loops
- Infinite Loops
- `break` Statement
- `continue` Statement
- Loop Control Variables
- Pattern Printing
- Menu-Driven Programming

Beginner Level Problems

Problem 1: Display Numbers from 1 to N

Difficulty: Beginner

Problem Statement

Accept a positive integer **N** from the user and display all numbers from **1 to N** using a `for` loop.

Problem 2: Sum of Natural Numbers

Difficulty: Beginner

Problem Statement

Accept a positive integer **N** and calculate the sum of the first **N** natural numbers.

Problem 3: Multiplication Table

Difficulty: Beginner

Problem Statement

Accept an integer and display its multiplication table from **1 to 10**.

Problem 4: Even and Odd Numbers

Difficulty: Beginner

Problem Statement

Accept a range of numbers and display all even numbers followed by all odd numbers within the given range.

Problem 5: Factorial Calculator

Difficulty: Beginner

Problem Statement

Accept a positive integer and calculate its factorial using a loop.

Problem 6: Reverse Counting

Difficulty: Beginner

Problem Statement

Display numbers from **N to 1** using both a `for` loop and a `while` loop.

Problem 7: Count Digits

Difficulty: Beginner

Problem Statement

Accept an integer and determine the total number of digits present in it.

Problem 8: Sum of Digits

Difficulty: Beginner

Problem Statement

Accept an integer and calculate the sum of all its digits.

Problem 9: Reverse a Number

Difficulty: Beginner

Problem Statement

Accept an integer and display its reverse.

Example:

Input:

```
12345
```

Output:

```
54321
```

Problem 10: Power Calculation

Difficulty: Beginner

Problem Statement

Accept a base and an exponent from the user and calculate the result without using any built-in mathematical functions.

Intermediate Level Problems

Problem 11: Prime Number Checker

Difficulty: Intermediate

Problem Statement

Accept an integer and determine whether it is a prime number.

Problem 12: Prime Numbers in a Range

Difficulty: Intermediate

Problem Statement

Accept two integers representing a range and display all prime numbers within that range.

Problem 13: Fibonacci Series

Difficulty: Intermediate

Problem Statement

Generate the Fibonacci sequence up to **N** terms.

Problem 14: Armstrong Number

Difficulty: Intermediate

Problem Statement

Accept an integer and determine whether it is an Armstrong number.

Problem 15: Palindrome Number

Difficulty: Intermediate

Problem Statement

Determine whether a given integer is a palindrome.

Problem 16: Perfect Number

Difficulty: Intermediate

Problem Statement

Accept an integer and determine whether it is a perfect number.

Problem 17: Least Common Multiple (LCM)

Difficulty: Intermediate

Problem Statement

Accept two positive integers and calculate their Least Common Multiple (LCM).

Problem 18: Greatest Common Divisor (GCD)

Difficulty: Intermediate

Problem Statement

Accept two positive integers and determine their Greatest Common Divisor (GCD) using an iterative approach.

Problem 19: Menu-Driven Calculator

Difficulty: Intermediate

Problem Statement

Develop a menu-driven calculator using a `do-while` loop.

Menu options:

1. Addition

2. Subtraction
3. Multiplication
4. Division
5. Exit

The program should continue until the user selects the **Exit** option.

Problem 20: Number Guessing Simulation

Difficulty: Intermediate

Problem Statement

Generate a predefined secret number in the program. Continuously prompt the user to guess the number until the correct value is entered.

Display appropriate hints such as:

- Too High
- Too Low
- Correct Guess

Advanced Level Problems

Problem 21: Pattern Printing – Right Triangle

Difficulty: Advanced

Problem Statement

Display the following pattern using nested loops:

```
*
**
***
****
*****
```

Problem 22: Floyd's Triangle

Difficulty: Advanced

Problem Statement

Generate Floyd's Triangle for **N** rows.

Example:

```
1
2 3
4 5 6
7 8 9 10
```

Problem 23: Pascal's Triangle

Difficulty: Advanced

Problem Statement

Generate Pascal's Triangle for the specified number of rows.

Problem 24: Student Result Processing

Difficulty: Advanced

Problem Statement

Accept the marks of **N** students.

Calculate and display:

- Highest Marks
- Lowest Marks
- Average Marks
- Number of Students Passed
- Number of Students Failed

Problem 25: ATM Menu Simulation

Difficulty: Advanced

Problem Statement

Design a menu-driven ATM application using a `do-while` loop.

The application should provide options such as:

- Balance Inquiry
- Deposit
- Withdraw
- Mini Statement
- Exit

Continue displaying the menu until the user selects **Exit**.

Challenge Problems

1. Print the following hollow square pattern for a given size.
2. Generate a pyramid and an inverted pyramid using nested loops.
3. Display all Armstrong numbers within a given range.
4. Simulate a traffic signal controller using loops and menu options.
5. Develop a simple text-based quiz application that allows multiple attempts.
6. Generate the Collatz (3n + 1) sequence for a given number.
7. Simulate a digital clock displaying hours, minutes, and seconds using nested loops.
8. Find all twin prime pairs within a specified range.
9. Implement the Euclidean Algorithm for GCD using both iterative and repeated subtraction approaches.
10. Create a console-based menu for unit conversions (temperature, distance, weight) using loops.

Real-World Applications

The concepts learned in this module can be applied to develop:

- ATM systems
- Banking transaction menus
- Billing systems
- Online examination portals
- Inventory management systems
- Payroll processing
- Student result analysis

- Data validation systems
- Password retry mechanisms
- Console-based business applications

Common Mistakes

Students should avoid the following errors:

- Creating infinite loops unintentionally.
- Incorrect initialization or update of loop control variables.
- Using the wrong loop type (`for`, `while`, or `do-while`) for the problem.
- Off-by-one errors in loop conditions.
- Incorrect placement of `break` and `continue` statements.
- Modifying the loop variable inside the loop body without justification.
- Failing to validate user input before entering loops.
- Improper nesting of loops resulting in incorrect output.
- Forgetting to reset variables inside nested loops.

Viva Questions

1. What is the purpose of iterative statements in Java?
2. Differentiate between `for`, `while`, and `do-while` loops.
3. When is a `do-while` loop preferred over a `while` loop?
4. Explain the role of initialization, condition, and update expressions in a `for` loop.
5. What is an infinite loop? How can it be avoided?
6. Differentiate between `break` and `continue`.
7. What are nested loops? Give practical examples.
8. Which loop is best suited for menu-driven applications? Why?
9. How can loops improve program efficiency and reduce code duplication?
10. What are common errors encountered while writing loop-based programs?

Expected Learning Outcomes

Upon successful completion of Module 3, students will be able to:

- Implement iterative solutions for repetitive computational tasks.
- Choose the appropriate loop construct for different programming scenarios.

- Solve mathematical and logical problems using loops.
- Develop menu-driven and interactive console applications.
- Generate complex patterns using nested loops.
- Write efficient, readable, and well-structured Java programs involving iterative control structures.

Instructor Note: Encourage students to first develop the algorithm or flowchart before writing the Java code. This practice improves logical reasoning and helps students understand how loop control variables and termination conditions influence program execution. For selected problems, ask students to compare solutions using `for`, `while`, and `do-while` loops to reinforce their understanding of each construct.